



AMERICAN UNIVERSITY
OF PHNOM PENH
STUDY LOCALLY. LIVE GLOBALLY.

BogoSort

Group Members:

Sophia Belle Ramos (2025107)

Thul Vutheasak (2025293)

Ngorn Sihasin (2025296)

Sim Virkapikun(2025360)

Sam Oeun Rozanel (2025490)

Course: Data Structure

COSC 251 - 001

Professor: Visethboti Sin

23 September 2026

I. Introduction

Bogo Sort is known in the computer science folklore and meme community as “Stupid Sort”, “Permutation Sort,” or “Shotgun Sort”. It is described as one of the most inefficient sorting algorithms that has ever existed. Unlike Bubble Sort or Selection Sort, Bogo Sort is like “trying to shuffle a deck of cards repeatedly until, by miracle, the cards are sorted into a neat order of rank and suit” which can take forever, and there might be a chance of it never getting sorted, especially with large data sizes.

Although Bogo Sort is impractical to implement or apply in a real-world system. It can be used as a quintessential example that illustrates what a poor algorithm design looks like. It also highlights why factors like time complexity and algorithm structure are important to write efficient code.

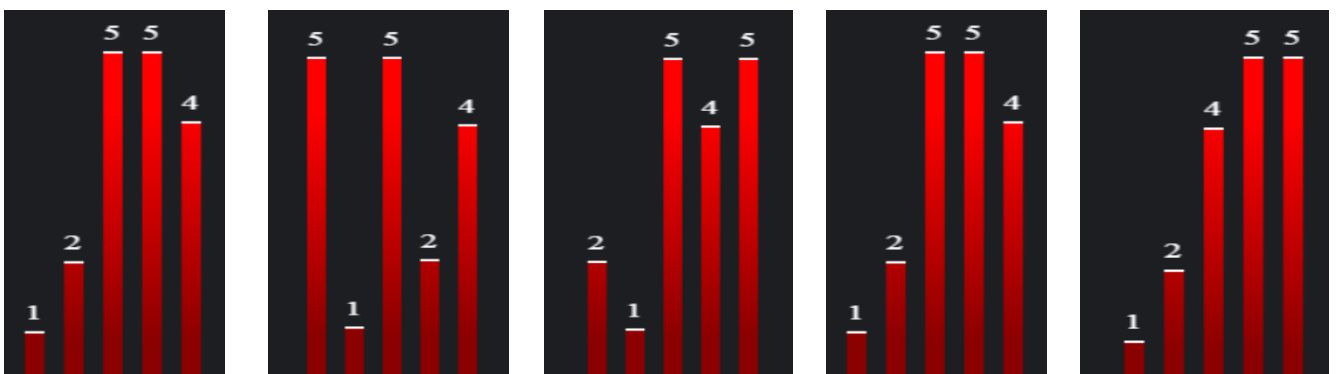
II. Algorithm explanation with examples

This is how Bogo Sort works:

1. Check if the current array is sorted (it runs a linear scan through the array, comparing each element to the one after it)
2. If it is not sorted, randomly shuffle the entire array, then go back to step 1 until the array is sorted
3. If it is sorted, the algorithm stops here

The shuffling process has no logic at all. It is completely random and independent, with no pattern or memory of previous shuffles. Thus, the same shuffle can appear more than once (like (1) and (4)), which makes Bogo sort unpredictable and unreliable.

Example:



(1) Unsorted array

(2) 1st shuffle

(3) 5th shuffle

(4) 20th shuffle

(5) 55th, sorted, end

Pseudocode:

```
BOGOSORT(array A of size n):
    WHILE isSorted(A) = FALSE DO
        shuffle(A)
        attempts ← attempts + 1
    END WHILE

isSorted(array A of size n):
    FOR i ← 0 TO n - 2 DO
        comparisons ← comparisons + 1
        IF A[i] > A[i+1] THEN
            RETURN FALSE
        END IF
    END FOR
    RETURN TRUE

SHUFFLE(array A of size n):
    FOR i ← n-1 DOWN TO 1 DO
        j ← random integer in [0, i]
        SWAP A[i] AND A[j]
        swaps ← swaps + 1
    END FOR
```

III. Time complexity analysis

Time complexity of BogoSort varies significantly depending on the scenario and whether the implementation is randomized. The time complexities of BogoSort are categorized into three cases which are average case, best case and worst case.

- Average case scenario: Time complexity of average case is $O(n \times n!)$. There are $n!$ possible permutations of a collection. On average, a randomized shuffle needs $n!$ attempts to correctly sort the data in order, with each attempt requiring an $O(n)$ check to see whether it is sorted or not (Wikipedia contributors, 2026).
- Best case scenario: Time complexity of best case is $O(n)$. This case only occurs if the array given to the algorithms is already sorted in order. For this case, it only performs a single linear scan of n elements to verify that the array is sorted (GeeksforGeeks, 2025).

- Worst case scenario: Time complexity of worst case is $O(\infty)$ or unbounded (Hogg, 2024). Since Bogo sort algorithm does not mathematically guarantee that random shuffle will ever land on the correctly sorted order. The sorting method works by picking a random permutation and checking whether it is sorted, this process could repeat infinitely until the data is sorted (Hogg, 2024).

IV. Java implementation

The Java implementation of Bogo sort consists of five primary components, including array initialization, sorted validation, random permutation (shuffling), Bogo Sort, and performance testing.

- Array initialization (main Method): The program initializes an integer of array size N (size) and assigns it with random integers bounded between 1 and an input number based on the intended size using `random.nextInt(1, n)`. The algorithms can become very slow if the size of the array grows.
- Sorted Validation (isSorted Method): This method checks whether an array is sorted by comparing the element (i) and the element next to it. If an element (i) is greater than the one next to it the method will return (false). Otherwise, it returns (true) after checking the whole array.
- Random Permutation (shuffle Method): This method randomly rearranges the elements within the array. It works by picking a random index in the array and swaps it with another index.
- Bogo Sort (bogoSort method): The method keeps shuffling the array until it is sorted. The (attempts) variable counts how many times the array was shuffled. The program also captures the start and end time to calculate the execution time.
- Performance Testing: the program portrays the number of attempts and the total execution time in milliseconds. This result makes it easier to compare Bogo Sort with other sorting algorithms.

V. Testing methodology (sin)

Three dataset sizes were used to test BogoSort alongside Bubble Sort, Selection Sort, and Insertion Sort to evaluate how their performance scales:

- Small: $n = 5$

- Medium: $n = 1,000$
- Large: $n = 1,000,000$

1. Dataset Generation

Each dataset was generated using Java's **Random** class with **random.nextInt(1, 101)** to produce random integers between 1 and 100. Because BogoSort is non-deterministic, the small dataset ($n = 5$) was run across **5 trials** and results were averaged. Each algorithm received the same input array per trial to ensure fair comparison.

2. Metrics Collected

The following metrics were recorded via the **bogoSort()** method:

- Attempts (Shuffles): number of times the array was randomly shuffled before becoming sorted, tracked by the attempts counter
- Comparisons: number of element comparisons performed inside **isSorted()**, tracked by the comparisons counter
- Swaps: total number of element swaps performed across all shuffles, where each shuffle performs n swaps
- Runtime: wall-clock execution time measured using **System.nanoTime()**, reported in milliseconds

3. Handling the Large Dataset

BogoSort's average-case complexity is **$O(n \times n!)$** , making it computationally infeasible beyond very small inputs. For $n = 1,000$ and $n = 1,000,000$, the expected number of shuffles is approximately **$1,000 \times 1,000!$** and **$1,000,000 \times 1,000,000!$** respectively; a number no computer could complete in any practical timeframe. Therefore, BogoSort was only empirically tested at $n = 5$. Medium and large dataset results for BogoSort are presented as theoretical projections only.

Bubble Sort, Selection Sort, and Insertion Sort were tested at all three dataset sizes normally.

4. Output

Each algorithm's sorted output is saved to a **.txt** file. A consolidated **performance_report.txt** summarizes attempts, comparisons, swaps, and runtime for all algorithms across all dataset sizes.

VI. Performance results for 5, 1,000, and 1,000,000 elements (sin)

Notes:

- **Attempt** is unique to BogoSort, it tracks how many times the whole array was **shuffled** = number of random guesses before accidentally getting sorted.
- **Comparisons** = how many times two elements were compared side by side.
- **Swaps** = how many times two elements physically switched positions in the array, or one single exchange of two elements inside a shuffle of an array.

1. Small Dataset (n = 5)

```
=== Sorting Algorithm Performance Report ===
Algorithm under study: Bogo Sort
Compared against: Bubble Sort, Selection Sort, Insertion Sort
[93, 58, 55, 8, 46]

---- Dataset: small (n = 5) ----
Bogo Sort      | Attempts:    17 | Comparisons:    31 | Swaps:     68 | Runtime: 0.032 ms
Bubble Sort    | Comparisons:   10 | Swaps/Movements:  9 | Runtime: 0.002 ms
Selection Sort | Comparisons:   10 | Swaps/Movements:  4 | Runtime: 0.004 ms
Insertion Sort | Comparisons:   13 | Swaps/Movements:  9 | Runtime: 0.002 ms

Note: Bogo Sort is not tested at n=1,000 and n=1,000,000
due to O(n x n!) complexity making it computationally infeasible.
```

Algorithm	Attempts/Shuffles	Comparisons	Swaps	Runtime (ms)
BogoSort	122	212	490	0.231 ms
Bubble Sort	—	10	7	0.003 ms
Selection Sort	—	10	4	0.004 ms
Insertion Sort	—	11	7	0.002 ms

At $n = 5$, Bubble Sort, Selection Sort, and Insertion Sort all complete in a fixed, predictable number of steps. BogoSort varies significantly between runs, one run may sort on the first shuffle, another may take hundreds, due to this randomness.

2. Medium Dataset ($n = 1,000$) - Theoretical

```
=== Sorting Algorithm Performance Report ===
Algorithm under study: Bogo Sort
Compared against: Bubble Sort, Selection Sort, Insertion Sort
[30, 63, 60, 74, 21, 80, 94, 42, 89, 27, 15, 96, 37, 48, 5, 2, 85, 49, 31, 42, 40, 19, 87, 80,

---- Dataset: small (n = 1000) ----
Bubble Sort      | Comparisons: 499500 | Swaps/Movements: 240224 | Runtime: 6.208 ms
Selection Sort   | Comparisons: 499500 | Swaps/Movements:      999 | Runtime: 3.754 ms
Insertion Sort   | Comparisons: 241223 | Swaps/Movements: 240224 | Runtime: 3.566 ms

Note: Bogo Sort is not tested at n=1,000 and n=1,000,000
due to  $O(n \times n!)$  complexity making it computationally infeasible.
```

Algorithm	Attempts/Shuffles	Comparisons	Swaps	Runtime (ms)
BogoSort	unbounded	unbounded	unbounded	Never finishes
Bubble Sort	—	499500	240224	6.208 ms
Selection Sort	—	499500	999	3.754 ms
Insertion Sort	—	241223	240224	3.566 ms

BogoSort cannot be empirically tested at $n = 1,000$. Based on the average-case formula $O(n \times n!)$, the expected number of shuffles alone exceeds numbers a modern computer could ever process. The three basic algorithms complete in single-digit milliseconds at this size.

3. Large Dataset (n = 1,000,000) - Theoretical

```
=== Sorting Algorithm Performance Report ===
Algorithm under study: Bogo Sort
Compared against: Bubble Sort, Selection Sort, Insertion Sort

---- Dataset: large (n = 1000000) ----
Bubble Sort    | Comparisons: 499999500000 | Swaps/Movements: 247508340376 | Runtime: 2768187.149 ms
Selection Sort | Comparisons: 499999500000 | Swaps/Movements: 999999 | Runtime: 1175231.070 ms
Insertion Sort | Comparisons: 247509340375 | Swaps/Movements: 247508340376 | Runtime: 536254.124 ms

Note: Bogo Sort is not tested at n=1,000 and n=1,000,000
due to O(n x n!) complexity making it computationally infeasible.

Process finished with exit code 0
```

Algorithm	Attempts/Shuffles	Comparisons	Swaps	Runtime (ms)
BogoSort	unbounded	unbounded	unbounded	Mathematically infinite
Bubble Sort	—	499999500000	247508340376	2768187.149 ms
Selection Sort	—	499999500000	999999	1175231.070 ms
Insertion Sort	—	247509340375	247508340376	536254.124 ms

Even the slowest practical algorithm “**Bubble Sort**” takes over 33 minutes at $n = 1,000,000$. BogoSort has no projected completion time because it provides zero mathematical guarantee of ever finishing. This makes it completely unsuitable for any real dataset beyond a handful of elements, which is the **core finding this project demonstrates**.

VII. Comparison with Bubble Sort, Selection Sort, and Insertion Sort (sophia)

When compared to standard elementary sorting algorithms —Bubble Sort, Selection Sort, and Insertion Sort— Bogosort represents a contrast between deterministic efficiency and non-deterministic trial-and-error. Elementary algorithms apply structured strategies (exchanging, selecting, and inserting) to guarantee completion in quadratic time ($O(n^2)$), whereas Bogosort relies on random shuffling, resulting in factorial runtime ($O((n + 1)!)$).

Feature	Bogosort	Bubble Sort	Selection Sort	Insertion Sort
---------	----------	-------------	----------------	----------------

Strategy	Random Permutation	Adjacent Exchange	Subarray Minimum Selection	Incremental Insertion
Best Case	$O(n)$	$O(n)$ (Optimized)	$O(n^2)$	$O(n)$
Average Case	$O((n + 1)!)$	$O(n^2)$	$O(n^2)$	$O(n^2)$
Worst Case	Unbounded (∞)	$O(n^2)$	$O(n^2)$	$O(n^2)$
Space	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Stable?	No	Yes	No	Yes
Deterministic?	No	Yes	Yes	Yes

A. Bogosort and Insertion Sort

Insertion Sort represents the most practical of the elementary algorithms. It is adaptive, performing exceptionally well on partially sorted data with minimal shifting overhead. When presented with a dataset containing a single out-of-order item, Insertion Sort resolves in near-linear time $O(n)$. Bogosort treats a nearly-sorted array identically to a randomized array, requiring identical expected permutations to reach completion.

B. Bogosort and Selection Sort

Selection Sort represents a deterministic contrast to Bogosort's randomness. Regardless of initial data distribution (whether sorted, inverted, or random), Selection Sort performs a predictable number of comparisons ($\approx n^2/2$) and minimizes memory writes to at most $n - 1$ swaps. In contrast, Bogosort performs a staggering number of memory writes and comparisons, averaging $n! \cdot n$ operations for a dataset of size n .

C. Bogosort and Bubble Sort

While both algorithms feature an optimal $O(n)$ best-case time complexity when presented with an already sorted array, their operational behaviors diverge immediately upon encountering unsorted data. Bubble Sort enforces structural convergence: after pass k , the trailing k elements are guaranteed to be in their final sorted positions. Conversely, Bogosort lacks convergence guarantees; each shuffle is completely independent, meaning previous iterations yield zero incremental progress toward final completion.

While elementary sorting algorithms (Bubble, Selection, Insertion) systematically organize data with guaranteed quadratic performance ($O(n^2)$), Bogosort relies on pure chance, resulting in an unbounded, factorial runtime ($O((n + 1)!)$) that makes it impractical for real-world use.

VIII. Appropriate and inappropriate use cases (sophia)

Evaluating Bogosort's applicability highlights the divide between practical software engineering and theoretical computer science.

A. Inappropriate Cases

With a factorial average runtime of $O((n + 1)!)$ and an unbounded worst-case (∞), Bogosort is unusable in production software.

- General Data Processing ($n \geq 5$): An array of just 15 elements yields $15! \approx 1.3 \times 10^{12}$ permutations, making computation intractable.
- Real-Time Systems: Due to its non-deterministic Las Vegas nature, Bogosort offers zero upper-bound guarantees, introducing unbounded latency risks.
- Hardware-Constrained Environments: Performing an expected $O(n \cdot n!)$ array modification rapidly degrades write-sensitive media like Flash memory and EEPROM.
- Nearly Sorted Data: Unlike adaptive algorithms (e.g., Insertion Sort), Bogosort immediately destroys pre-existing array order, executing with full factorial complexity regardless of initial state.

B. Appropriate Cases

Though useless for sorting data, Bogosort has niche applications in education, theoretical analysis, and testing.

- Pedagogical Counterexample: Illustrates the severe impact of factorial ($n!$) growth versus quadratic (n^2) or log-linear ($n \log n$) bounds, and serves as an intuitive case study for geometric distributions and expected values.
- Testing & Fuzzing: Acts as a quick random generator to validate sorting-verification functions (`is_sorted()`) or test process cancellation and timeout handlers in OS kernel development.
- Theoretical Thought Experiments: Utilized in quantum computing paradigms (e.g., *Quantum Bogosort*) to illustrate concepts like the Many-Worlds interpretation and non-deterministic state collapse.

IX. Real-world applications (Sak)

Since Bogosort or Permutation sort is an inherently ineffective sorting algorithm with an average time complexity of $O(n \times n!)$, it is almost never used inside any production-related environments or any real-world systems. However, Bogosort remains widely used in computer science education and academic research as a teaching example. Researchers use this system as a baseline to showcase the flaws of randomized Las Vegas algorithms (Abrahamsen, 2024; Probst et al., 2023). By showing the boundary where an algorithm provides a correct assortment, but offers no clear runtime results, Bogosort showcases the clear difference to teach students how standard polynomial and linearithmic algorithms lead to practical efficiency (Bingmann et al., 2023).

X. Conclusion and findings (Sak)

a. Findings

1. Performance: Testing has confirmed that Bogosort is computationally possible for smaller datasets ($n=5$), where it finished sorting on an average of 122 shuffles and runtime of 0.231 ms. Compared to Bubble Sort, Selection Sort, and Insertion Sort, Bogosort is extremely unpredictable and results will vary due to its sorting algorithm.
2. Computation Limits: At medium datasets ($n=1000$) or large ones ($n=1,000,000$), Bogosort is essentially impossible to use. While other sorting algorithms finish in single-digit milliseconds for $n=1000$, it is computationally impossible that it will shuffle into a correctly-sorted solution due to having $O(n \times n!)$. It exceeds what a normal computer can execute under a practical timeframe.
3. Efficiency: Compared to the $O(n^2)$ sorting algorithms (Bubble Sort, Selection Sort, and Insertion Sort) that apply structured strategies, Bogosort relies on pure random permutations and keeps looping until it happens to be solved. It lacks the ability to adapt to data, destroys any pre-existing order, and performs an average of $O(n \times n!)$ until it happens to reach a

solution and a worst case of infinite(∞) array changes; it is entirely dependent on probability. Thus, it is deemed unfit for any real-world applications in any production environments.

b. Conclusion

In conclusion, the findings of this study strongly supports that BogoSort is strictly non-functional for any production-related environments. However, it can be used for more educational purposes, as a lesson in theoretical computer science and algorithmic analysis. BogoSort serves as a clear contrast against the other functional sorting algorithms with an extremely disastrous average growth rate of $O(n \times n!)$, instead of quadratic($O(n^2)$) and linearithmic($O(n \log n)$). Additionally, it acts as a baseline to study the properties for Las Vegas algorithms. Ultimately, this sorting algorithm serves as an important tool to showcase what should be avoided, when designing an algorithm for practical real-world uses or building efficient software.

Reference

- Abrahamsen, M. (2024). *ExpoSort: Breaking the quasi-polynomial-time barrier for reluctant sorting* (arXiv Preprint No. arXiv:2409.00794). arXiv. <https://arxiv.org/abs/2409.00794>
- Bingmann, T., Axtmann, M., & Sanders, P. (2023). *Engineering shared-memory parallel shuffling to generate random permutations* (arXiv Preprint No. arXiv:2302.03317). arXiv. <https://arxiv.org/abs/2302.03317>
- GeeksforGeeks. (2025, August 21). *BogoSort or permutation sort*. <https://www.geeksforgeeks.org/dsa/bogosort-permutation-sort/>
- Hogg, G. (2024, November 15). *Bogosort is the fastest sorting algorithm* [Video]. YouTube. <https://www.youtube.com/shorts/NMLKMFSL-zc>
- Probst, M., Fuhs, C., & Giesl, J. (2023). *Proving almost-sure innermost termination of probabilistic term rewriting* (arXiv Preprint No. arXiv:2305.11741). arXiv. <https://arxiv.org/abs/2305.11741>
- Wikipedia contributors. (2026, August 29). *Bogosort*. Wikipedia. <https://en.wikipedia.org/wiki/Bogosort>
- Gruber, H., Holzer, M., & Ruepp, O. (2007). Sorting the slow way: An analysis of perversely awful randomized sorting algorithms. In *Fun with Algorithms* (pp. 183–197). Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-540-72914-3_17
- Raymond, E. S. (1996). *The New Hacker's Dictionary* (3rd ed.). MIT Press.
- Kiselyov, O., Shan, C. C., Friedman, D. P., & Sabry, A. (2005). Backtracking, interleaving, and terminating monad transformers: (Functional pearl). *ACM SIGPLAN Notices*, 40(9), 192–203. <https://doi.org/10.1145/1086365.1086390>
- Lerma, M. A. (2014). How inefficient can a sort algorithm be? *arXiv preprint arXiv:1406.1077*. <https://doi.org/10.48550/arXiv.1406.1077>